



Projet de formation - Option B

Monde de D v - MVP d'un r seau social pour d veloppeurs

Annexe : Analyse des besoins frontend

Auteur : Emmanuel Hesry

Version : 1.0.0

Date : 31/08/2026

Statut : version finale

Sommaire

Annexe : Analyse des besoins frontend.....	1
Sommaire.....	1
1. Objet et p�rim�tre.....	2
2. Utilisateurs et t�ches prioritaires.....	2
3. Parcours fonctionnels.....	2
3.1 Parcours principal.....	2
3.2 Cas alternatifs et retours attendus.....	3
4. Fonctionnalit�s et crit�res d'acceptation.....	4
5. �crans, composants et adaptation responsive.....	4
6. Contrat frontend-backend et s�curit�.....	5
7. Validation, qualit� et limites.....	6
7.1 Validation des formulaires.....	6
7.2 Qualit�, accessibilit� et tests.....	6
7.3 Limites assum�es du MVP.....	6

1. Objet et périmètre

MDD est un réseau social destiné aux développeurs. L'interface permet à un visiteur de créer un compte puis, à un membre authentifié, de suivre des thèmes, consulter le fil associé, publier des articles, commenter et gérer son profil.

L'objectif est de proposer ces parcours de façon simple, responsive et cohérente avec les maquettes fournies. Le périmètre reste celui du MVP : aucune administration, recherche, fonctionnalité sociale avancée, édition ou suppression d'article ou de commentaire n'est prévue.

2. Utilisateurs et tâches prioritaires

Utilisateur	Objectif	Tâches prioritaires
Visiteur	Accéder à la communauté	Consulter l'accueil, s'inscrire, se connecter.
Membre	Suivre des sujets techniques	Consulter les thèmes, s'abonner, lire son fil.
Membre	Partager et échanger	Créer un article, consulter son détail, ajouter un commentaire.
Membre	Gérer son compte	Modifier son profil et se désabonner d'un thème.

3. Parcours fonctionnels

3.1 Parcours principal

- Le visiteur arrive sur la page d'accueil et choisit l'inscription ou la connexion.
- Après authentification, le membre consulte la liste des thèmes et s'abonne à ceux qui l'intéressent.
- Il ouvre le fil des articles issus de ses abonnements et choisit l'ordre chronologique.
- Il consulte le détail d'un article, le commente ou publie son propre article.
- Depuis son profil, il met à jour ses informations et gère ses abonnements.

3.2 Cas alternatifs et retours attendus

Situation	Comportement attendu
Route privée sans session	Redirection vers la connexion avant l'accès à la page demandée.
Formulaire incomplet ou invalide	Envoi bloqué et message associé au champ concerné.
Erreur de l'API	Message compréhensible sans exposition de détail technique.
Article absent	Écran dédié avec possibilité de revenir au fil.
Menu mobile ouvert	Fermeture possible avec Échap ou un clic hors du panneau.

4. Fonctionnalités et critères d'acceptation

Fonctionnalité	Critères d'acceptation observables
Inscription et connexion	Les formulaires demandent les informations nécessaires, empêchent l'envoi invalide et ouvrent une session après une réponse API valide.
Déconnexion et session	La navigation affiche les actions du membre connecté ; la déconnexion ferme la session et les routes privées sont protégées.
Thèmes et abonnements	Chaque carte indique l'état d'abonnement ; l'action met à jour l'état affiché ; le désabonnement est disponible depuis le profil.
Fil	Seuls les articles des thèmes suivis sont chargés ; le tri chronologique peut être inversé ; chaque carte ouvre le détail.
Article et commentaire	Le membre choisit un thème, saisit un titre et un contenu pour publier ; le détail affiche les commentaires et permet d'en ajouter un.
Profil	Le membre visualise et modifie son nom d'utilisateur, son adresse e-mail et, s'il le souhaite, son mot de passe.
Responsive et navigation	Les écrans fonctionnent sur desktop et mobile ; le logo rejoint le fil ; le menu mobile est utilisable au clavier.

5. Écrans, composants et adaptation responsive

Les routes publiques sont /, /login et /register. Les routes /posts, /posts/create, /posts/:postId, /topics et /profile nécessitent une session. Une route inconnue mène à une page dédiée.

Les écrans sont organisés par domaines Angular : auth, topic, post et user. Des composants partagés couvrent notamment l'en-tête, les liens de navigation, les formulaires, les messages d'erreur, le bouton de retour et les icônes.

Sur desktop, la navigation est directement visible. Sur mobile, elle est proposée dans un panneau

ouvert par le bouton de menu. Les mêmes liens fonctionnels sont utilisés dans les deux présentations. Le logo est un lien permanent vers le fil. Les formulaires disposent de libellés, de messages de validation et d'un focus visible.

Les dix-sept captures desktop et mobile, dont le menu mobile ouvert, sont remises dans une archive ZIP séparée conformément au format de livraison retenu.

6. Contrat frontend-backend et sécurité

Les services Angular typés communiquent avec l'API REST sous /api. Les détails de schémas et les exemples de requêtes sont publiés dans Swagger UI. Le contrat couvre l'authentification, le profil courant, les thèmes et abonnements, les articles et les commentaires.

Domaine	Routes utilisées par le frontend
Authentification	POST /api/auth/register, POST /api/auth/login, POST /api/auth/logout
Session et profil	GET /api/users/me, PATCH /api/users/me
Thèmes	GET /api/topics, POST /api/topics/{topicId}/subscription, DELETE /api/topics/{topicId}/subscription
Articles et commentaires	GET /api/posts?sort=asc desc, POST /api/posts, GET /api/posts/{postId}, POST /api/posts/{postId}/comments

Le JWT est conservé dans un cookie HttpOnly et n'est donc pas lisible par Angular. Au chargement, le frontend restaure l'état de session avec GET /api/users/me. Avant toute requête qui modifie l'état, il initialise la protection CSRF ; le cookie XSRF-TOKEN fournit la valeur transmise dans l'en-tête X-XSRF-TOKEN.

7. Validation, qualité et limites

7.1 Validation des formulaires

Les écrans de saisie reposent sur les formulaires réactifs Angular. À l'inscription, le mot de passe doit contenir de 8 à 72 caractères, dont une majuscule, une minuscule, un chiffre et un caractère spécial. Les règles client améliorent le retour immédiat ; le backend applique également ses propres contrôles.

7.2 Qualité, accessibilité et tests

- TypeScript strict, composants standalone et styles SCSS.
- Services HTTP typés ; une façade n'est introduite que lorsqu'elle orchestre réellement état, actions et navigation.
- Tests unitaires et d'intégration Vitest, scénarios Cypress ciblés et vérifications API Bruno.
- Navigation responsive, libellés de formulaires, retour d'erreur associé et interactions clavier du menu mobile.

Les résultats mesurés, les commandes d'exécution et les rapports générés sont détaillés dans l'annexe « Rapport de tests et de couverture ».

7.3 Limites assumées du MVP

Le MVP ne prévoit ni rôle administrateur, ni recherche, ni likes, ni édition ou suppression de contenus, ni notifications, pièces jointes ou sous-commentaires. Ces exclusions maintiennent un périmètre cohérent avec la mission et ne constituent pas des anomalies.